Evaluating the File Transfer Service

Three experiments: packet loss, file size, and scaling under load

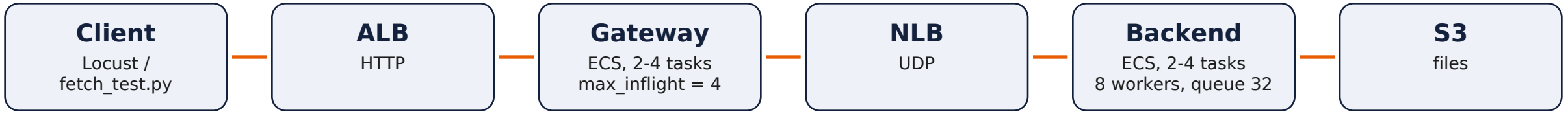
CS6650 Building Scalable Distributed Systems - Midterm Mastery Part 2

Kunal Juvvala

Experiment Presentation - November 2026

The service and how I tested it

Deployed on AWS (us-west-2) with the repo's deploy scripts



Key facts from reading the code

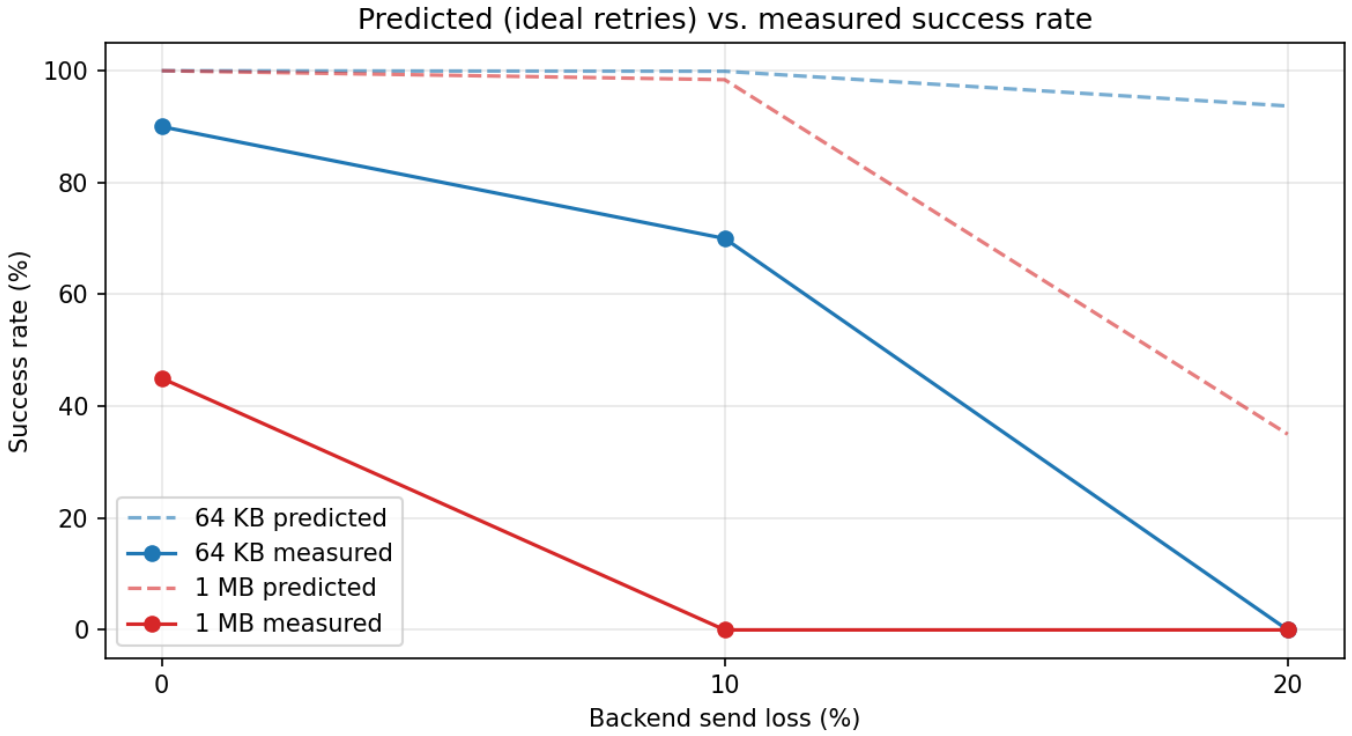
- **64-byte chunks:** every chunk is its own UDP datagram, so 1 MB = 16,384 datagrams. Max file 4 MiB = 65,536 chunks.
- **2 s floor:** the client decides the server is done only after 2 s of silence, so every download takes at least ~2 s.
- **Retries:** 5 OPEN attempts, then up to 5 rounds of RETY (one message per missing chunk). Out of rounds = 504.
- **Scaling:** gateway on requests/min per task (target 100); backend on average CPU (target 60%).

Method

- **One variable at a time:** everything else held at defaults.
- **Exp 1:** vary send loss 0 / 10 / 20% (receive loss fixed at 0), 64 KB and 1 MB files.
- **Exp 2:** vary file size 1 KB to 4 MB + one over the limit, at 0% loss.
- **Exp 3:** Locust step load 5 -> 15 -> 40 -> 5 users on a 1 KB file; ECS events for scaling.
- **Predictions first,** then measured; scripts and raw CSVs are in the repo.

Experiment 1: How much packet loss can it survive?

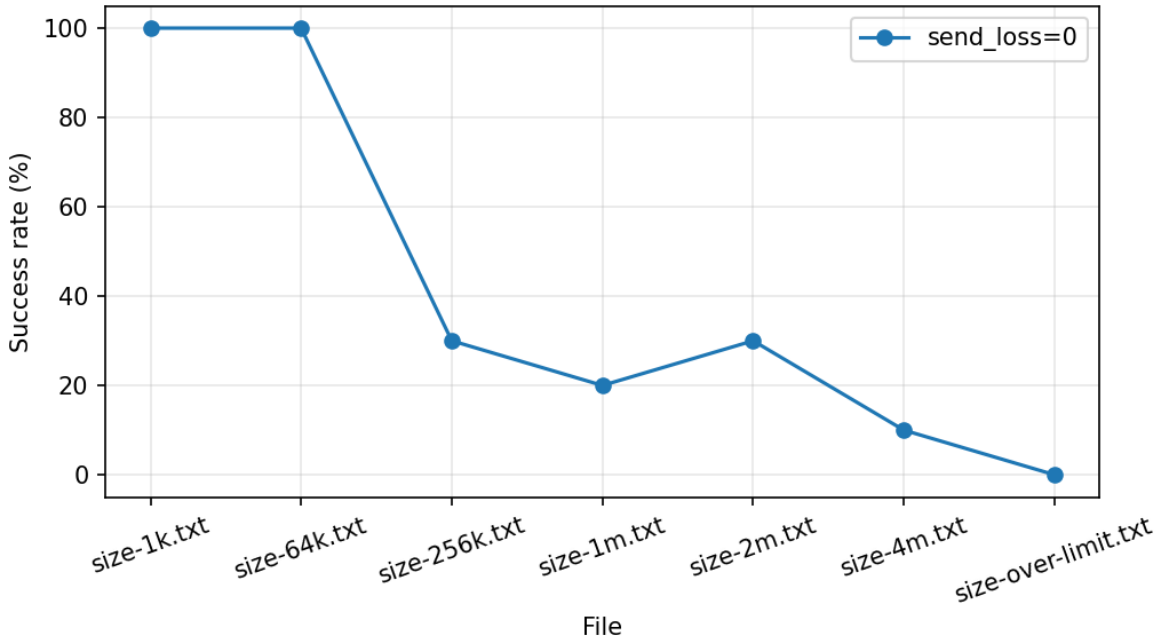
Varied backend send loss; fixed retries, scaling, and receive loss (0%)

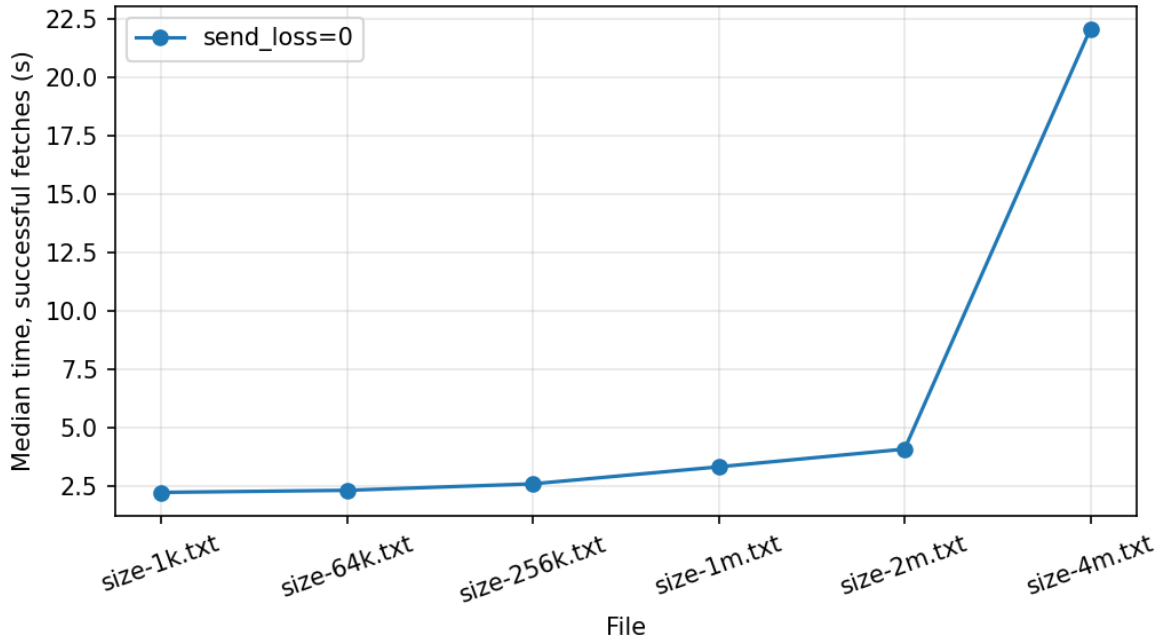


- **Far worse than predicted.** The dashed lines assume retries work like the first send: (1 - p^6)^N.
- **10% loss:** 1 MB never succeeded (predicted ~98%). 64 KB worked 7/10, but took 10.9 s instead of 2.4 s.
- **20% loss:** even 64 KB never succeeded (predicted ~94%).
- **Even 0% loss:** only 45% of 1 MB downloads worked.
- **Every failure:** a 504 after 13-19 s, once all 5 retry rounds were used up.

Experiment 2: How does file size matter?

Varied file size at 0% configured loss; 10 tries each





- **Small files are fine:** 1 KB and 64 KB worked 10/10 at ~2.3 s, almost all of it the 2 s quiet timeout.
- **256 KB and up mostly fail (10-30%),** with no configured loss at all. Successes are fast (2.6-4 s); failures are slow (15-36 s).
- **Over the 4 MiB limit:** 0/10, reported as a misleading 504 "did not respond" after ~20 s, not "file too large".

Why retries collapse: what the code shows

transfer-server/protocol/server.go

1. Burst

OPEN sends every chunk at once (16,384 datagrams for 1 MB). Bursts overflow UDP buffers, so some chunks are lost even at 0% loss.

2. Retry flood

The client sends one RETY per missing chunk, all at once: hundreds or thousands of messages.

3. Tiny queue

The backend has 8 workers and a 32-slot queue. When it is full, new messages are dropped ("Overloaded").

4. Expensive retry

Each RETY re-downloads the WHOLE file from S3 and re-chunks it to send ONE 64-byte chunk.

5. Give up

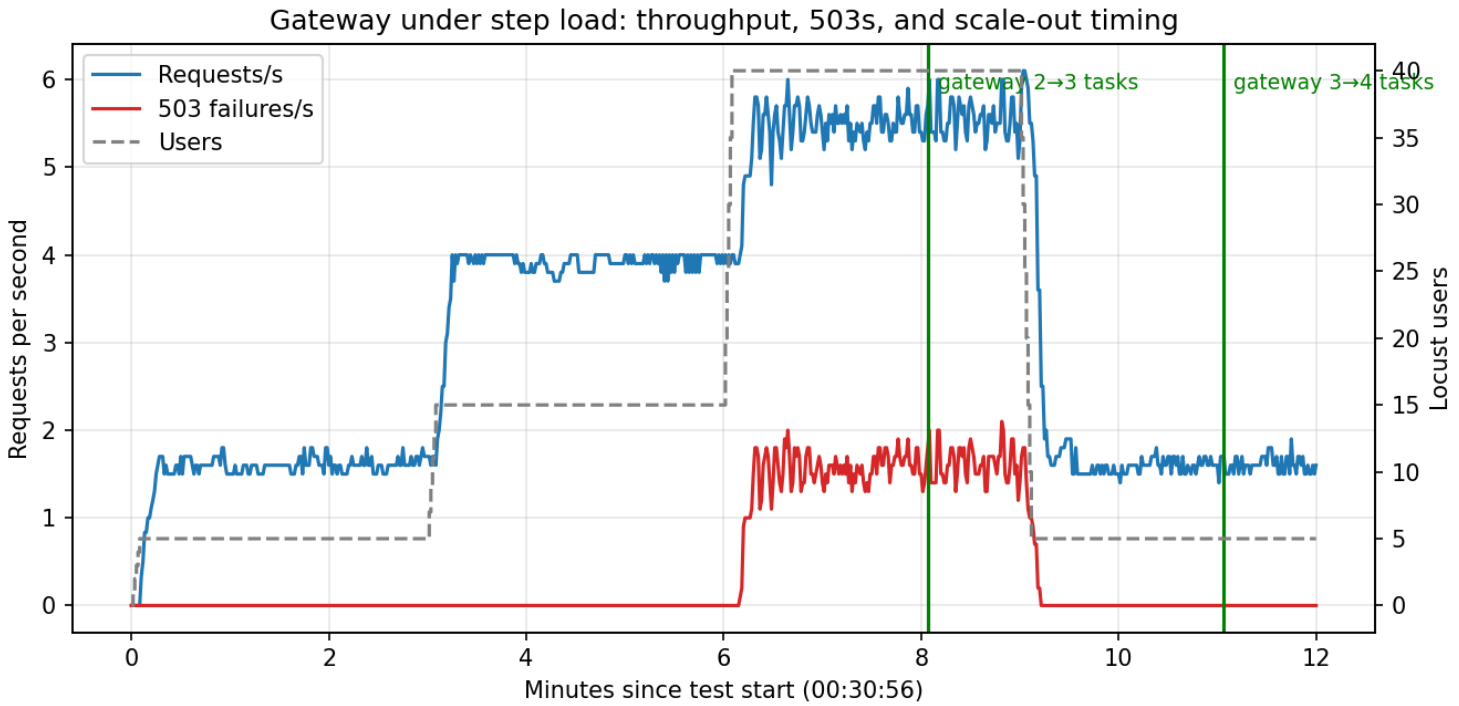
Each round recovers only a few chunks. After 5 rounds: 504.

Result: the reliability mechanism amplifies load exactly when the system is struggling.

Retry work grows with (missing chunks x file size), which explains both Experiment 1 and Experiment 2.

Experiment 3: Scaling under load

Locust step load on a 1 KB file, 0% loss, default scaling; 2,279 requests in 12 minutes



- **Ceiling predicted and found:** at 15 users throughput went flat at exactly 4.0 req/s = 2 tasks x 4 in-flight / ~2 s each.
- **40 users:** 28% of requests got 503 "gateway busy" (5 s queue wait). p95 rose to 7 s.
- **Scaling lagged:** 2 -> 3 tasks came ~2 min after the spike; 3 -> 4 came after the load had already dropped.
- **Backend never scaled:** stayed at 2 tasks; little CPU work, so the 60% CPU target was never reached.

What the experiments reveal

Strengths, weaknesses, and what I would change

Findings

- **Retries are the weak point:** fine with no loss and small files, but moderate loss or files over ~256 KB cause mostly 504s.
- **Gateway throughput is set by waiting, not work:** max_inflight x the 2 s quiet timeout caps each task at ~2 req/s.
- **Reactive scaling is too slow for spikes:** users absorb the spike as 503s for minutes.
- **CPU is the wrong backend metric:** the backend drops requests from its queue long before CPU rises.

Recommended changes

- **Retries:** batch many chunk IDs per RETY; cache the file between OPEN and RETY; larger backend queue.
- **Sending:** pace the initial burst; use larger chunks; clear "file too large" error.
- **Gateway:** raise max_inflight (tasks are idle while waiting) or shorten the 2 s timeout.
- **Scaling:** lower the requests-per-target value and scale-out cooldown; scale the backend on a load signal, not CPU.

Limitations and next steps

- **Small samples:** 10-20 tries per setting to fit time and budget; percentages are rough, but the trends are large and consistent across experiments.
- **Not varied:** receive loss, retry counts, the 2 s timeout, max_inflight, workers, and cooldowns.
- **Short load steps:** 3 minutes is short for target tracking; scale-in was not observed.
- **Root cause from code reading:** the next step is backend logs counting "Overloaded" drops per download to confirm it directly.
- **Next experiments:** rerun Exp 1 with a batched RETY fix, and Exp 3 with max_inflight = 16.